

lecture1

May 26, 2026

1 Quantum Computing Bootcamp

1.1 Last Time

1. Brief historical context (RSA, Feynman, Shor)
2. Introduction to Qubits and Gates

1.2 Today: Lecture 1

1. Technical dive into qubits and gates
2. Entanglement
3. Quantum teleportation

2 1977: The Cryptographic Milestone of RSA

1977: Rivest, Shamir, and Adleman introduce RSA, the gold standard of public-key cryptography.

- Multiplying two large primes is easy.
- Factoring their product appears hard for classical computers.
- That asymmetry lets strangers communicate securely over an insecure channel.

Ron Rivest

Adi Shamir

Leonard Adleman

2.0.1 RSA in a Nutshell

Choose large primes p and q (a secret key).

$$N = pq$$

Publish N (the public key). A typical size of N used today is 2048-bits.

N can be used to encrypt data, (p, q) is used to decrypt data. Since the key used to encrypt data is different from the one used to decrypt, we call this form of exchange “asymmetric”.

The security of RSA is derived from the computational difficulty of solving the factorization problem. An attacker who can efficiently recover p and q from N can break RSA.

3 1981: Feynman's Idea

1981: Richard Feynman gives a keynote lecture at the First Conference on the Physics of Computation at MIT.

His central observation:

- Nature is quantum.
- Classical simulation of quantum systems appears to require exponential resources.
- Maybe the right computer for quantum physics is itself quantum.

A classical computer stores bits and updates them with logic gates. A quantum computer would store quantum states and update them with physical operations that obey the laws of quantum mechanics.

4 1994: Shor's Algorithm

1994: Peter Shor presents a polynomial-time quantum algorithm for integer factoring and discrete logarithms.

- RSA depends on factoring being hard.
- Diffie-Hellman and elliptic-curve cryptography depend on discrete log variants being hard.
- Shor shows both are efficiently solvable on an ideal fault-tolerant quantum computer.

The cryptographic impact comes from the reduction: **factoring** $\rightarrow$ **period finding** $\rightarrow$ **QFT**.

Caveat: Quantum computation does not promise speedup for every task. Some problems have special structure that quantum mechanics can exploit more directly than classical probability can. For these problems, quantum computation can offer speedup.

5 What this means for cryptography

Quantum computers do **not** immediately break all cryptography.

They do threaten widely deployed public-key systems that use quantum-vulnerable encryption schemes (e.g. RSA) once sufficiently large, reliable, fault-tolerant quantum devices exist.

This is why the field now cares about:

- Quantum algorithms such as Shor and Grover
- Quantum-safe/post-quantum cryptography
- Quantum cryptographic protocols such as BB84
- Realistic resource estimates and error correction

6 What makes quantum computers so powerful?

Three ingredients show up again and again:

Superposition: amplitudes can represent combinations of classical states.

Interference: amplitudes can add or cancel, amplifying useful outcomes and suppressing bad ones.

Entanglement: multi-qubit states can contain correlations with no classical bit-by-bit description.

The art of quantum algorithms is arranging these effects so measurement is likely to reveal the answer to an otherwise hard-to-solve problem (e.g. factoring).

7 Classical Bits vs. Quantum Bits

A classical bit is either

$$0 \quad \text{or} \quad 1$$

A qubit is a state

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where $\alpha, \beta \in \mathbb{C}$ and

$$|\alpha|^2 + |\beta|^2 = 1$$

The probabilities of measuring 0 or 1 are $|\alpha|^2$ and $|\beta|^2$, respectively.

8 The Bloch Sphere

The most general expression for a pure single-qubit state is

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

The angles (θ, ϕ) place the state on a unit-sphere, called the Bloch sphere.

- North pole: $|0\rangle$
- South pole: $|1\rangle$
- Equator: balanced superpositions

This gives us a geometric way to think about single-qubit states and gates.

Qiskit lets us build circuits, simulate quantum states, draw circuit diagrams, and submit jobs to quantum computer backends.

```
[ ]: # Bloch sphere examples: direct initialization
from math import sqrt
from IPython.display import display, Markdown
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

states = [
    (r"$|0\rangle$", Statevector([1, 0])),
    (r"$|1\rangle$", Statevector([0, 1])),
    (r"$(|0\rangle + |1\rangle)/\sqrt{2}$", Statevector([1/sqrt(2), 1/
↪sqrt(2)])),
```

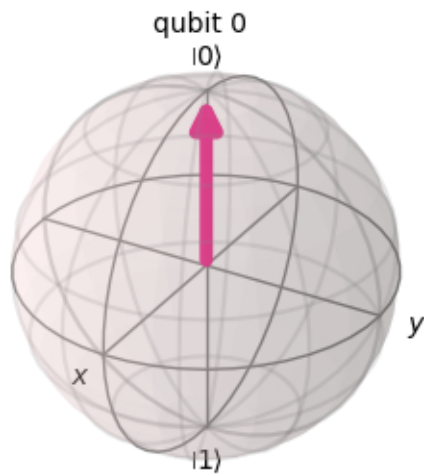
```

]

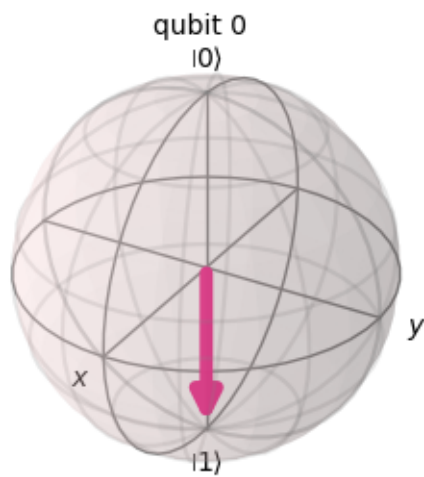
for label, state in states:
    display(Markdown(f"### {label}"))
    display(plot_bloch_multivector(state, figsize=(3.0, 3.0), title=""))

```

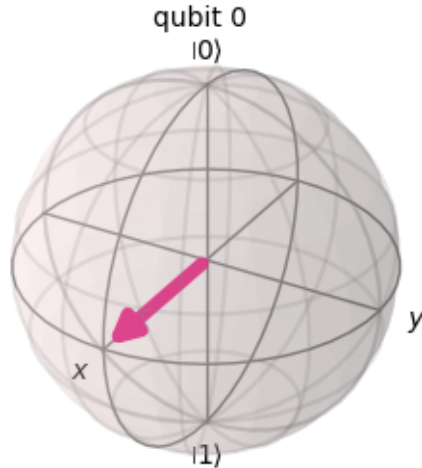
8.0.1 $|0\rangle$



8.0.2 $|1\rangle$



8.0.3 $(|0\rangle + |1\rangle)/\sqrt{2}$



9 Reading from Qubits: Measurement

Measurement converts quantum information into classical information.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Measuring in the computational basis gives:

- outcome 0 with probability $|\alpha|^2$
- outcome 1 with probability $|\beta|^2$

After measurement, the state is no longer the original superposition. It has collapsed to the observed basis state.

```
[ ]: # A one-qubit measurement circuit
from math import sqrt
from qiskit import QuantumCircuit

measure_qc = QuantumCircuit(1, 1)           # 1 qubit and 1 classical bit
# state
measure_qc.initialize([1/sqrt(2), 1/sqrt(2)], 0) # directly prepare (|0> + |1>) / sqrt(2)
measure_qc.measure(0, 0)                     # read the qubit into a classical bit
measure_qc.draw(output="mpl")
```

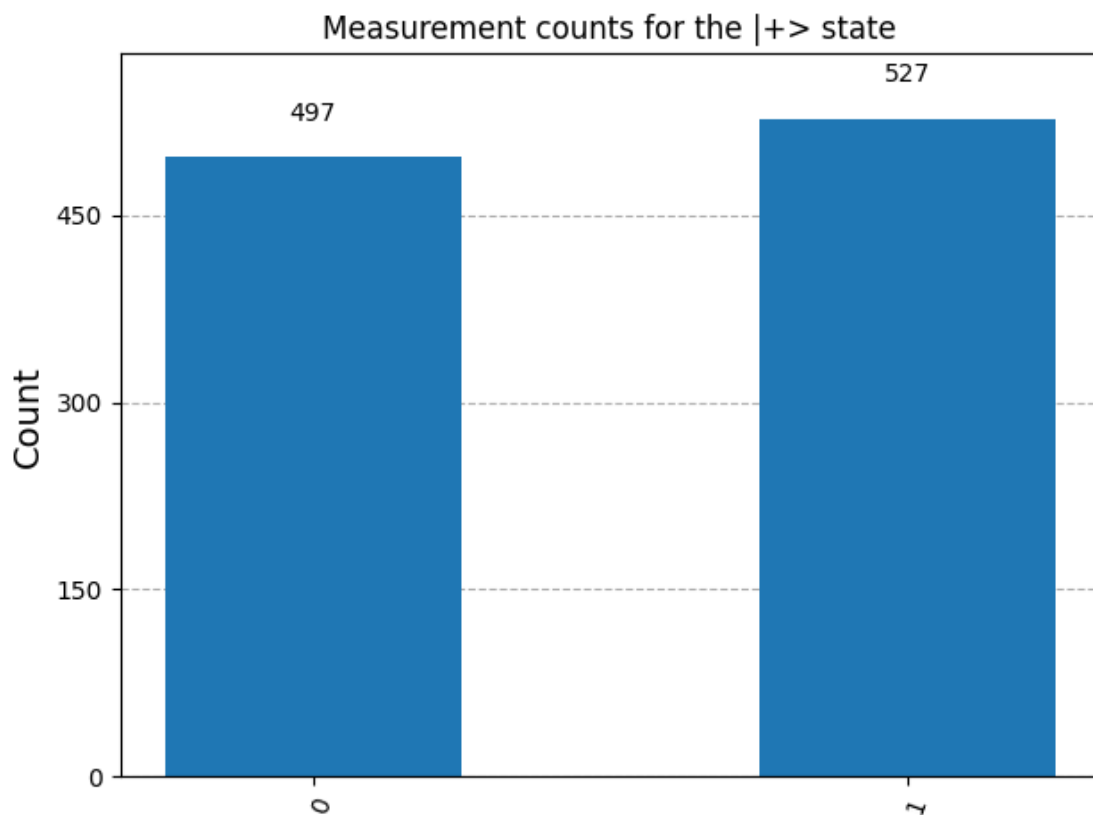
```
[ ]:
```



```
[3]: # Simulate measurement counts and show a histogram
from math import sqrt
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram

plus_state = Statevector([1/sqrt(2), 1/sqrt(2)])
counts = plus_state.sample_counts(shots=1024)
plot_histogram(counts, title="Measurement counts for the |+> state")
```

[3]:



10 Writing to Qubits: Quantum Gates

10.0.1 Classical Gates and Logical Circuits

Classical computation is built from gates that act on bits: inputs are definite 0 or 1 values, and outputs are definite 0 or 1 values.

Common examples:

- **NOT:** flips one bit, $x \mapsto \neg x$
- **AND:** outputs 1 only if both inputs are 1
- **OR:** outputs 1 if at least one input is 1
- **XOR:** outputs 1 if the inputs differ

A logical circuit wires these gates together so the output of one gate becomes the input to another. Quantum circuits keep the circuit picture, but replace bits with qubits and Boolean operations with unitary operations.

10.0.2 From Bits to Qubits

Classical gates update definite bit strings. Quantum gates update **amplitudes**.

A one-qubit state is

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1.$$

A valid closed-system quantum gate must preserve total probability, so it is represented by a **unitary** matrix U satisfying

$$U^\dagger U = I.$$

10.0.3 Some preliminary notes about quantum gates

- Quantum gates are reversible/unitary operations.
- Multi-qubit states live in tensor-product spaces.
- A small collection of one-qubit gates plus an entangling two-qubit gate is enough to build general quantum circuits.

10.0.4 Qiskit Setup

This notebook is already executed, so the diagrams and histograms are visible without live coding. The cells remain runnable if you want to modify examples.

```
[2]: import os
os.environ.setdefault("MPLCONFIGDIR", "/private/tmp/mpl")
os.environ.setdefault("XDG_CACHE_HOME", "/private/tmp/cache")

import numpy as np
import matplotlib.pyplot as plt
from math import pi
from IPython.display import display
```

```

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit.providers.basic_provider import BasicSimulator
from qiskit.quantum_info import Statevector, Operator, partial_trace, \
    state_fidelity
from qiskit.visualization import plot_histogram, plot_bloch_multivector

backend = BasicSimulator()
SHOTS = 2048

plt.rcParams.update({"figure.figsize": (7, 4), "axes.titlesize": 14})

def run_counts(circuit, shots=SHOTS):
    compiled = transpile(circuit, backend)
    return backend.run(compiled, shots=shots).result().get_counts()

def measured_copy(circuit, n_qubits=None):
    n = n_qubits or circuit.num_qubits
    measured = circuit.copy()
    measured.measure_all()
    return measured

def show_state(circuit, label="statevector"):
    sv = Statevector.from_instruction(circuit)
    print(label)
    display(sv.draw("text"))
    return sv

```

10.1 One-Qubit Gates

A one-qubit gate is a 2×2 unitary matrix acting on the amplitude vector

$$\begin{bmatrix} \alpha & \beta \end{bmatrix}.$$

Common one-qubit gates:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

The Pauli gates X, Y, Z gates are axis flips/rotations on the Bloch sphere. The Hadamard gate maps computational-basis states into balanced superpositions.

11 Example: The Hadamard Gate

The Hadamard gate turns computational-basis states into balanced superpositions.

For $|0\rangle$:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle$$

For $|1\rangle$:

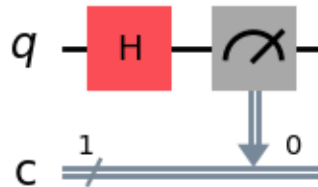
$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

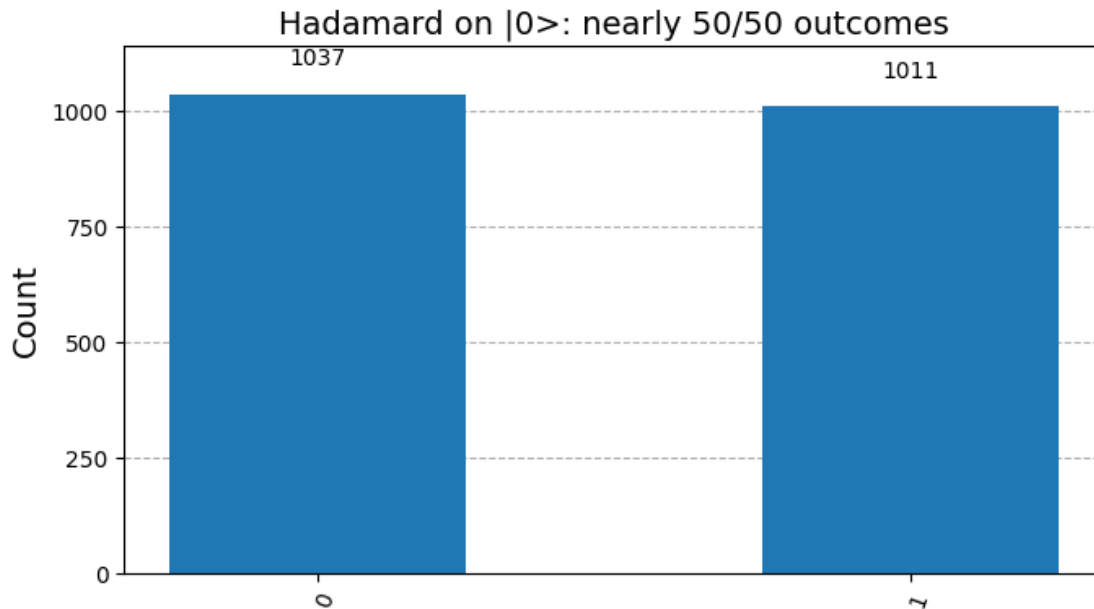
So H creates a 50/50 measurement distribution, but assigns a different **relative phase**. That phase is invisible in one direct measurement, but crucial when amplitudes later interfere.

Also: $H^2 = I$.

Note: The states $|+\rangle$ and $|-\rangle$ are each a superposition of the states $|0\rangle$ and $|1\rangle$. Superposition does **not** mean the qubit has become a random classical bit. If it were just random, applying another H would not deterministically restore $|0\rangle$.

```
[ ]: qc = QuantumCircuit(1, 1)
      qc.h(0)
      qc.measure(0, 0)
      display(qc.draw("mpl"))
      counts = run_counts(qc)
      plot_histogram(counts, title="Hadamard on |0>: nearly 50/50 outcomes")
```





12 Single-qubit gates as rotations

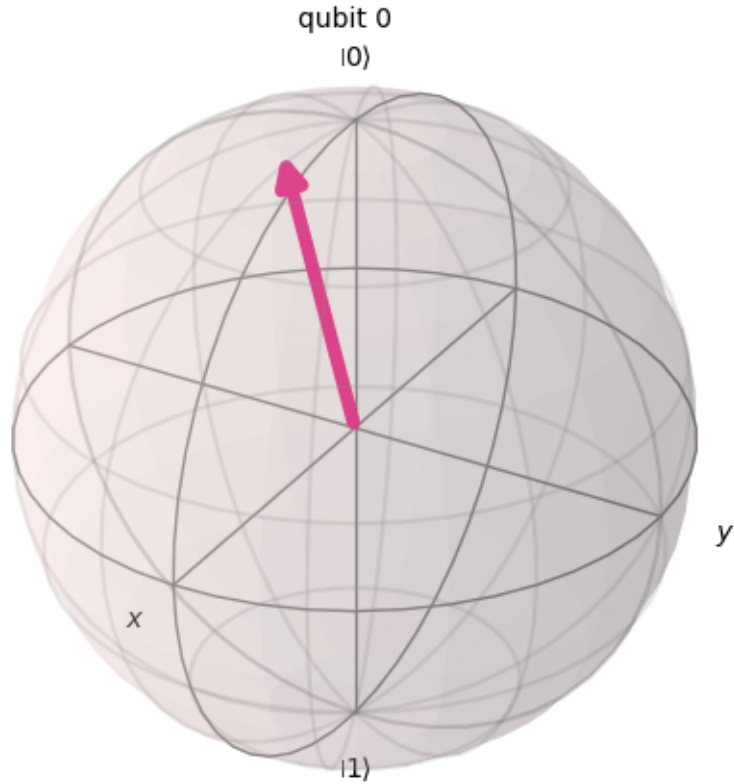
On the Bloch sphere, many single-qubit gates have a clean geometric meaning.

- X : rotation by π around the x-axis
- Y : rotation by π around the y-axis
- Z : rotation by π around the z-axis
- $R_x(\theta), R_y(\theta), R_z(\theta)$: rotations by chosen angles

```
[ ]: # Visualizing a single-qubit rotation
from math import pi
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

rot_qc = QuantumCircuit(1)
rot_qc.ry(-pi / 3, 0) # rotate |0> toward |1> around the y-axis
rot_qc.rz(pi / 4, 0)  # then rotate phase around the z-axis

state = Statevector.from_instruction(rot_qc)
plot_bloch_multivector(state)
```



12.0.1 Phase Gates: S and T

S and T are **phase gates**. They leave computational-basis probabilities alone, but they change the relative phase between $|0\rangle$ and $|1\rangle$:

$$S|0\rangle = |0\rangle, \quad S|1\rangle = i|1\rangle,$$

$$T|0\rangle = |0\rangle, \quad T|1\rangle = e^{i\pi/4}|1\rangle.$$

So on a state like $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, they rotate the state around the Bloch sphere's z -axis.

So what? Phase is invisible if we immediately measure in the $\{|0\rangle, |1\rangle\}$ basis, but later gates can turn differences in phase into differences in probability. We will see this later when constructing algorithms that rely on quantum interference of amplitudes.

12.1 Multi-Qubit States

For two qubits, basis states are tensor products:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle.$$

A general two-qubit state is

$$|\psi\rangle = a_{00}|00\rangle + a_{01}|01\rangle + a_{10}|10\rangle + a_{11}|11\rangle.$$

The state vector has four amplitudes; n qubits have 2^n amplitudes (one for each of 2^n basis states):

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$$

This exponential state space is a crucial feature of quantum computation!

12.1.1 Tensor Products of States

A two-qubit basis state is built by stacking products of amplitudes. In matrix notation,

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

The tensor product rule is

$$\begin{bmatrix} a_0 \\ a_1 \end{bmatrix} \otimes \begin{bmatrix} b_0 \\ b_1 \end{bmatrix} = \begin{bmatrix} a_0 b_0 \\ a_0 b_1 \\ a_1 b_0 \\ a_1 b_1 \end{bmatrix}.$$

Example:

$$|0\rangle \otimes |1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 0 \\ 1 \cdot 1 \\ 0 \cdot 0 \\ 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = |01\rangle.$$

This ordering is $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

If qubit A is $|a\rangle = a_0|0\rangle + a_1|1\rangle$ and qubit B is $|b\rangle = b_0|0\rangle + b_1|1\rangle$, then

$$|a\rangle \otimes |b\rangle = a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle.$$

Not every two-qubit state can be factored this way. Those non-factorable states are entangled.

12.1.2 Tensor Products of One-Qubit Matrices

Gates tensor together the same way, but each entry of the first matrix multiplies the entire second matrix.

If

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix},$$

then

$$A \otimes B = \begin{bmatrix} aB & bB \\ cB & dB \end{bmatrix} = \begin{bmatrix} ae & af & be & bf \\ ag & ah & bg & bh \\ ce & cf & de & df \\ cg & ch & dg & dh \end{bmatrix}.$$

Example: apply X to the first qubit and I to the second:

$$X \otimes I = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

Check on a state:

$$(X \otimes I)|01\rangle = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = |11\rangle.$$

12.1.3 CNOT: Controlled-NOT

CNOT flips the target bit if the control bit is 1. In the ordered basis

$$|\text{control target}\rangle = \{|00\rangle, |01\rangle, |10\rangle, |11\rangle\},$$

it acts as

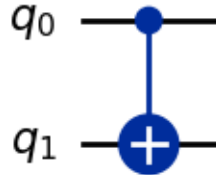
$$|c, t\rangle \mapsto |c, t \oplus c\rangle,$$

where $\oplus$ is XOR: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

So $|10\rangle \mapsto |11\rangle$ and $|11\rangle \mapsto |10\rangle$.

```
[ ]: qc = QuantumCircuit(2)
      qc.cx(0, 1)
      display(qc.draw("mpl"))
      print(Operator(qc).data)
```



```
[[1.+0.j 0.+0.j 0.+0.j 0.+0.j]
 [0.+0.j 0.+0.j 0.+0.j 1.+0.j]
 [0.+0.j 0.+0.j 1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j 0.+0.j 0.+0.j]]
```

12.1.4 Controlled-Z and SWAP

Controlled-Z applies a phase only to $|11\rangle$:

$$CZ = \text{diag}(1, 1, 1, -1).$$

The SWAP gate exchanges two qubits:

$$|a, b\rangle \mapsto |b, a\rangle,$$

and in the basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$,

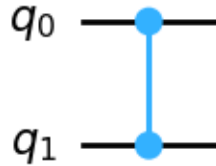
$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

On hardware, compilers often insert SWAPs when two logical qubits need to interact but are not physically connected.

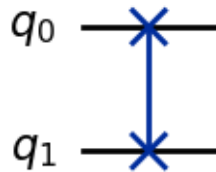
```
[ ]: cz = QuantumCircuit(2)
      cz.cz(0, 1)
      sw = QuantumCircuit(2)
```

```
sw.swap(0, 1)
print("CZ")
display(cz.draw("mpl"))
print("SWAP")
display(sw.draw("mpl"))
```

CZ



SWAP



12.1.5 Making a Bell State

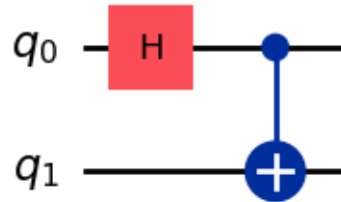
Start with $|00\rangle$.

1. Apply H to qubit 0: $|00\rangle \mapsto (|00\rangle + |10\rangle)/\sqrt{2}$
2. Apply CNOT with qubit 0 as control and qubit 1 as target:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

```
[ ]: bell = QuantumCircuit(2)
bell.h(0)
bell.cx(0, 1)
```

```
display(bell.draw("mpl"))
sv_bell = show_state(bell, "Bell state")
```



Bell state

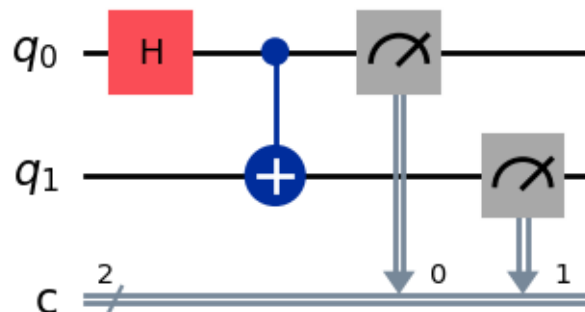
```
[0.70710678+0.j,0.      +0.j,0.      +0.j,0.70710678+0.j]
```

12.1.6 Entanglement in Measurement Data

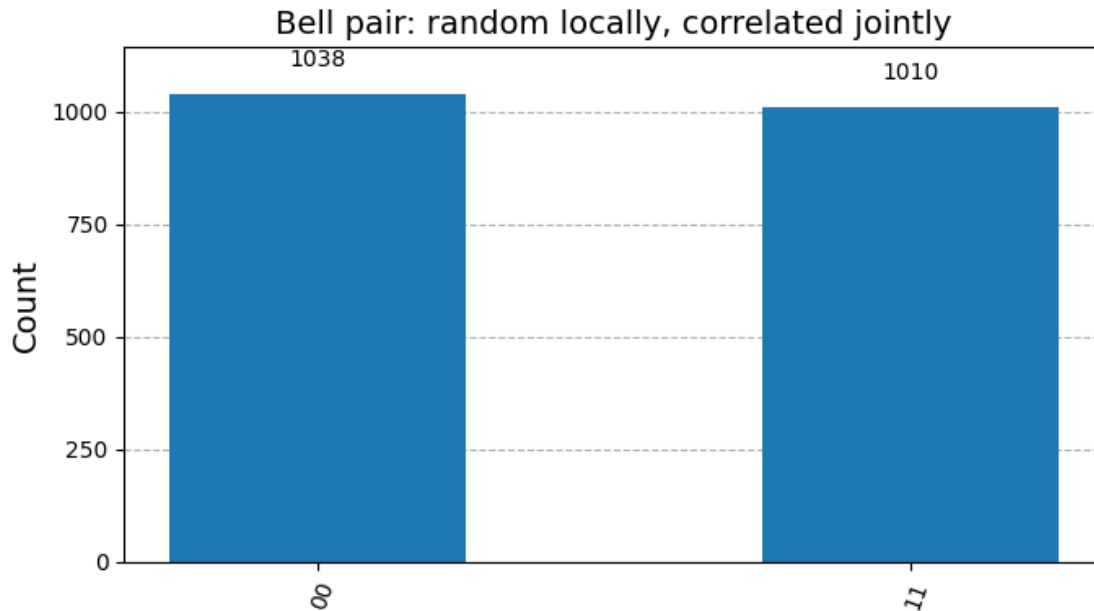
Each qubit alone looks random. Together, the outcomes are perfectly correlated.

That is the key signature in the computational basis for $|\Phi^+\rangle$: only 00 and 11 appear.

```
[8]: bell_meas = QuantumCircuit(2, 2)
bell_meas.h(0)
bell_meas.cx(0, 1)
bell_meas.measure([0, 1], [0, 1])
display(bell_meas.draw("mpl"))
plot_histogram(run_counts(bell_meas), title="Bell pair: random locally,
↪correlated jointly")
```



[8]:



12.1.7 Correlation Is Not the Whole Story

The Bell state and the product state $|++\rangle$ are both built using superposition, but they are not the same kind of state.

Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

Product state:

$$|++\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \otimes \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right).$$

Expanding the tensor product gives

$$|++\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

That extra $|01\rangle$ and $|10\rangle$ amplitude is the giveaway: $|++\rangle$ factors into two separate one-qubit states, while the Bell state cannot.

```
[13]: product_state = QuantumCircuit(2)
      product_state.h(0)
      product_state.h(1)
```

```

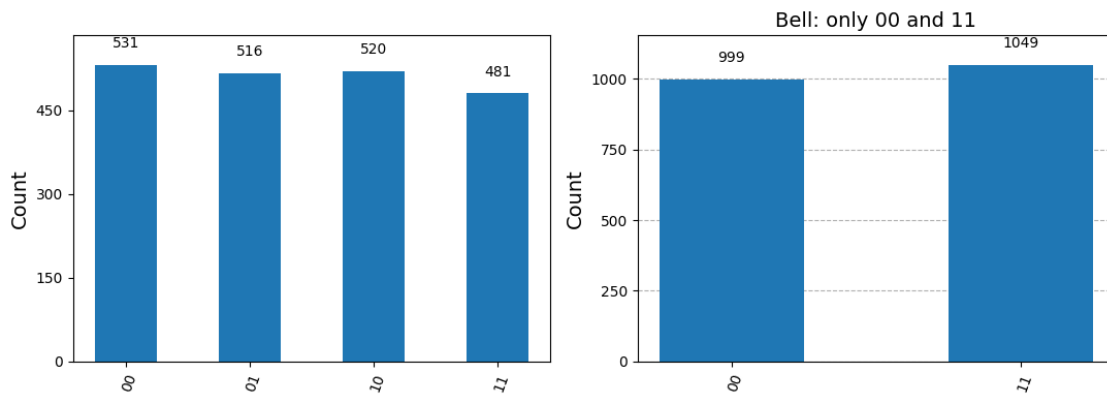
bell_state = QuantumCircuit(2)
bell_state.h(0)
bell_state.cx(0, 1)

# print("|++> statevector")
# display(Statevector.from_instruction(product_state).draw("text"))
# print("Bell statevector")
# display(Statevector.from_instruction(bell_state).draw("text"))

product = QuantumCircuit(2, 2)
product.compose(product_state, inplace=True)
product.measure([0, 1], [0, 1])

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
plot_histogram(run_counts(product), ax=axes[0], title="|++>: all four outcomes")
plot_histogram(run_counts(bell_meas), ax=axes[1], title="Bell: only 00 and 11")
plt.tight_layout()
# fig

```



12.1.8 Four Bell States

The four maximally entangled two-qubit Bell states are

$$\begin{aligned}
|\Phi^+\rangle &= \frac{|00\rangle + |11\rangle}{\sqrt{2}}, & |\Phi^-\rangle &= \frac{|00\rangle - |11\rangle}{\sqrt{2}}, \\
|\Psi^+\rangle &= \frac{|01\rangle + |10\rangle}{\sqrt{2}}, & |\Psi^-\rangle &= \frac{|01\rangle - |10\rangle}{\sqrt{2}}.
\end{aligned}$$

They form a basis for two-qubit space.